## 1. Trace the Stack Operations

| Step | Character | Operation | Stack after operation |
|---|---|---|---|
| 1 | { | Push | { |
| 2 | [ | Push | { [ |
| 3 | ( | Push | { [ ( |
| 4 | ) | Peek, Pop | { [ |
| 5 | ] | Peek, Pop | { |
| 6 | } | Peek, Pop | Empty |

## Task 3 – Display the Stack Execution Trace

| Character | Operation | Stack after operation |
|---|---|---|
| { | PUSH { | { |
| [ | PUSH [ | { [ |
| ( | PUSH ( | { [ ( |
| ) | PEEK (), POP ( | { [ |
| ] | PEEK [], POP [ | { |
| } | PEEK {}, POP { | Empty |

## Task 4 – Test the Program

| No. | Input | Expected output | Your output |
|---|---|---|---|
| 1 | () | Valid | Valid |
| 2 | ()[]{} | Valid | Valid |
| 3 | {[()]} | Valid | Valid |
| 4 | ({[]}) | Valid | Valid |
| 5 | {[(])} | Invalid | Invalid |
| 6 | ([)] | Invalid | Invalid |
| 7 | {[()] | Invalid | Invalid |
| 8 | {[()]}] | Invalid | Invalid |
| 9 | Empty input | Valid | Valid |
| 10 | ((a+b)*[c-d]) | Valid | Valid |

## C. Guide questions

a. Which delimiter caused the mismatch?
The closing bracket ] caused the mismatch.

b. Which opening delimiter was at the top of the stack at that point?
The opening parenthesis ( was at the top of the stack.

c. Why should the program reject the sequence?
The program should reject the sequence because the closing bracket ] does not match the most recently opened delimiter (. The stack requires delimiters to close in the reverse order in which they were opened.

**Task 6 – Explain the Stack Operations**

a. Push – Why is an opening delimiter pushed onto the stack?
An opening delimiter is pushed onto the stack because the program needs to remember it until its matching closing delimiter appears. The stack keeps the most recently opened delimiter at the top.

b. Peek – Why must the program examine the top element before deciding whether a closing delimiter matches?
The program must examine the top element because the most recently opened delimiter must be closed first. Using peek() lets the program check the delimiter without removing it immediately.

c. Pop – Why is the matching opening delimiter removed from the stack?
The matching opening delimiter is removed because it has already been correctly closed. The program can then check the next opening delimiter underneath it.

d. isEmpty – Why must the stack be empty after the entire sequence has been processed?
The stack must be empty because every opening delimiter should have a matching closing delimiter. If any opening delimiter remains, it was never closed, so the sequence is invalid.

**Guide Questions**

1. What is the LIFO principle? Explain how it applies to the stack used in this activity.
LIFO means Last-In, First-Out. It means that the last element added to the stack is the first element removed. In this activity, the most recently opened delimiter must be matched and removed before the program can process the earlier opening delimiters.

2. Consider the sequence { [ ( ) ] }. What is the contents of the stack immediately before ) is processed? What happens to the stack after ) is processed? Explain why.
Before ) is processed, the stack contains { [ (. The opening parenthesis ( is at the top. The program checks it using peek() and then removes it using pop() because it matches ). Afterward, the stack contains { [.

3. Why must the program use the top element of the stack when checking a closing delimiter? What problem would occur if the program searched for any matching opening delimiter instead?
The program must check the top element because delimiters must be properly nested. If it searches for any matching opening delimiter, it might accept incorrectly ordered delimiters. For example, ([)] has the same number of opening and closing parentheses and brackets, but the order is incorrect.

4. Why is the sequence ([)] invalid? Trace the stack until the program detects the error. Identify the two delimiters that do not properly match.
The sequence is invalid because the closing parenthesis ) appears while the opening bracket [ is still at the top of the stack.

| Step | Character | Operation | Stack after operation |
|---|---|---|---|
| 1 | ( | Push | ( ( |
| 2 | [ | Push | [ ( [ |
| 3 | ) | Peek | [; mismatch ( [ |

The two delimiters that do not properly match at the error are the opening bracket [ and the closing parenthesis ). The program rejects the sequence because [ must be closed before (.

5. Consider the sequence {[(. There is no closing delimiter. Why should the program report this sequence as invalid even though no mismatched closing delimiter was encountered?
The sequence is invalid because the stack still contains { [ ( after all characters have been processed. These opening delimiters do not have matching closing delimiters. The program must check whether the stack is empty at the end.

6. A student pushes opening delimiters and pops the stack for every closing delimiter without checking whether the stack is empty or comparing the delimiters. Explain at least two problems.
First, popping an empty stack can cause an error because there is no element to remove. Second, the program may accept mismatched delimiters because it does not check whether the popped opening delimiter matches the current closing delimiter. As a result, incorrect sequences could be reported as valid.

7. Question numbering in the document
The uploaded document moves from question 6 to question 8. No question 7 is shown, so there is no question 7 to answer.
8. Why can a stack be used to solve the matching-delimiter problem? Explain the relationship between nested delimiters and the LIFO behavior of a stack.
A stack is useful because nested delimiters must be closed in reverse order. For example, in {[()]}, the parenthesis closes first, followed by the bracket, and then the brace. This follows the LIFO principle of a stack.

9. Compare using a stack with simply counting the number of opening and closing delimiters. Why is a stack necessary for checking the order and nesting of delimiters?
Counting only checks how many opening and closing delimiters exist. It does not check whether they are in the correct order. For example, ([)] contains matching numbers of parentheses and brackets but is invalid. A stack checks the order and nesting by comparing each closing delimiter with the most recent unmatched opening delimiter.

10. Suppose the program is modified to support another pair of delimiters, such as < >. What changes would need to be made to the delimiter-checking logic?
I would update the isOpening() method to recognize < and the isClosing() method to recognize >. I would also update the isMatch() method so that < matches >. This would allow the program to check the additional delimiter pair using the same stack operations.

**AI Disclosure Statement**

1. Name of AI tool
OpenAI ChatGPT (GPT-6).

2. Task performed by the AI tool
The AI tool assisted in drafting the Java source code, explaining stack operations, preparing stack traces, checking test cases, and writing answers to the laboratory guide questions.

3. Prompts used
“Help me to understand this activity the Matching-Delimiters-Stack, the codes and how it works”

4. Extent of AI use
AI was used substantially to assist with drafting the program, explanations, test results, and written answers. I am responsible for reviewing, testing, and understanding the final submission.

5. Human verification
I will review the generated code and explanations, run the Java program in Eclipse, compare the actual results with the expected results, and correct any errors before submitting the activity.

6. Why and how AI was used
I used AI as a learning aid to understand how a stack works and how it can check matching delimiters. It helped me organize the laboratory answers and understand the purpose of push(), peek(), pop(), and isEmpty().

7. Estimated percentage of work assisted by AI
My estimated percentage of this work assisted by AI in 75%.

8. Changes made to the AI-generated output
I will verify the Java program in Eclipse, make corrections if needed, add my own student information, capture actual screenshots, record the program's actual results, and add my own public GitHub repository link.

9. What I learned from using the tool
I learned that a stack follows the LIFO principle and can be used to check whether delimiters are properly matched and nested. I also learned why the program must check the top of the stack before removing an element and why the stack must be empty at the end for a sequence to be valid.